

# 实验八实验报告

Huji-tye

2025 年 6 月 27 日

## 1 流水段 CPU 设计实验

### 1.1 原理图

采用经典五段流水线 CPU 设计，流水段为 IF、ID、EX、M、WB. CPU 设计原理图如下：

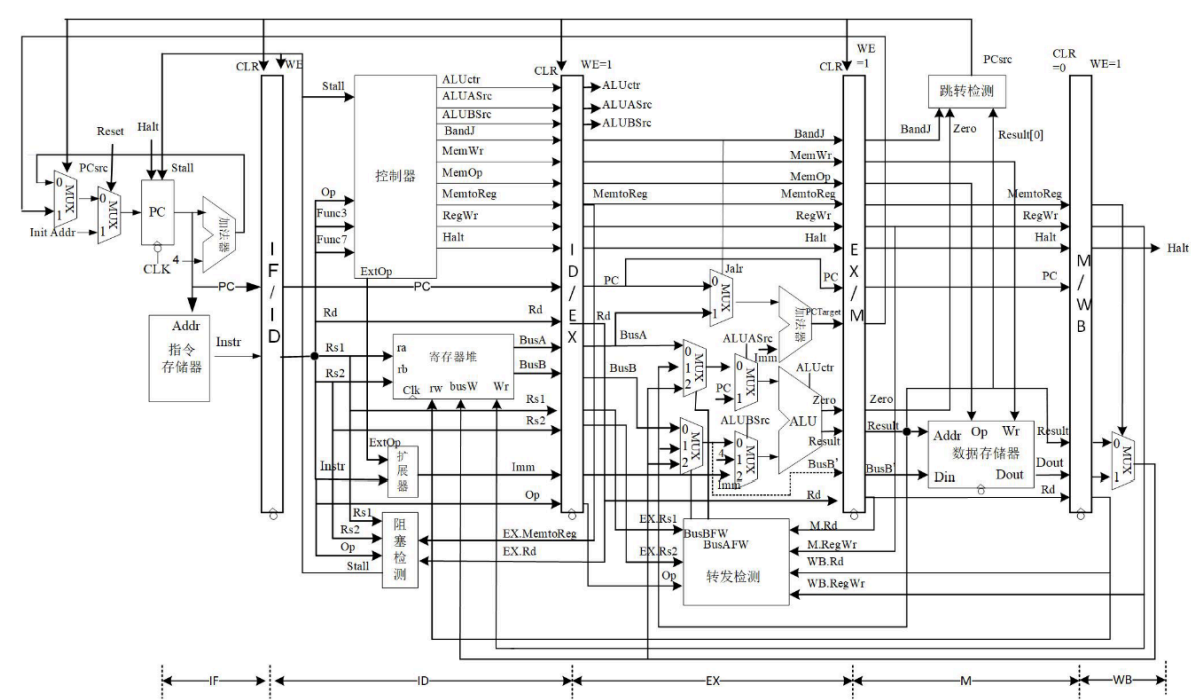


Figure 1: 流水线 CPU 设计原理图

## 1.2 电路图

### 1.2.1 转发检测模块

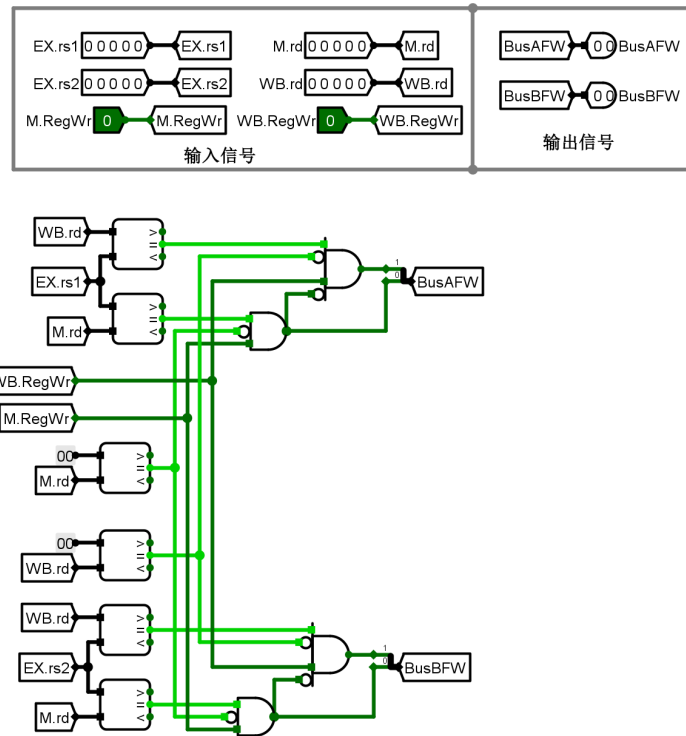


Figure 2: 转发检测模块电路图

### 1.2.2 阻塞检测模块

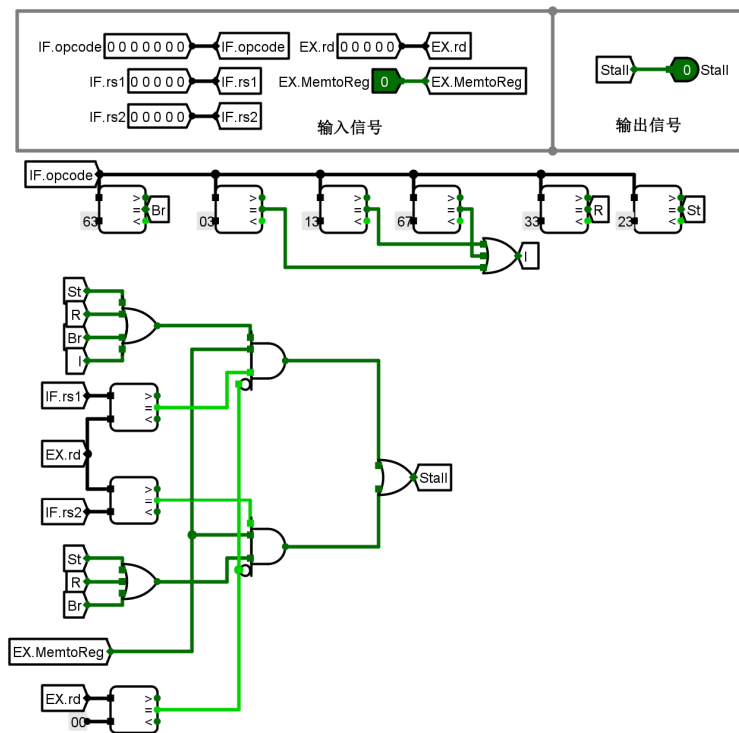


Figure 3: 阻塞检测模块电路图

### 1.2.3 跳转检测模块

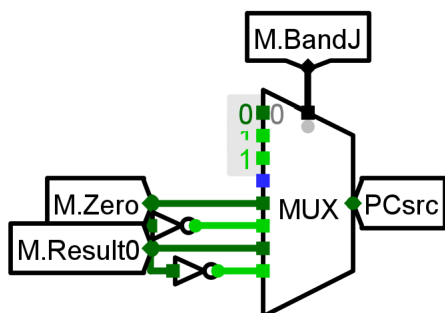


Figure 4: 跳转检测模块电路图

### 1.2.4 计数器

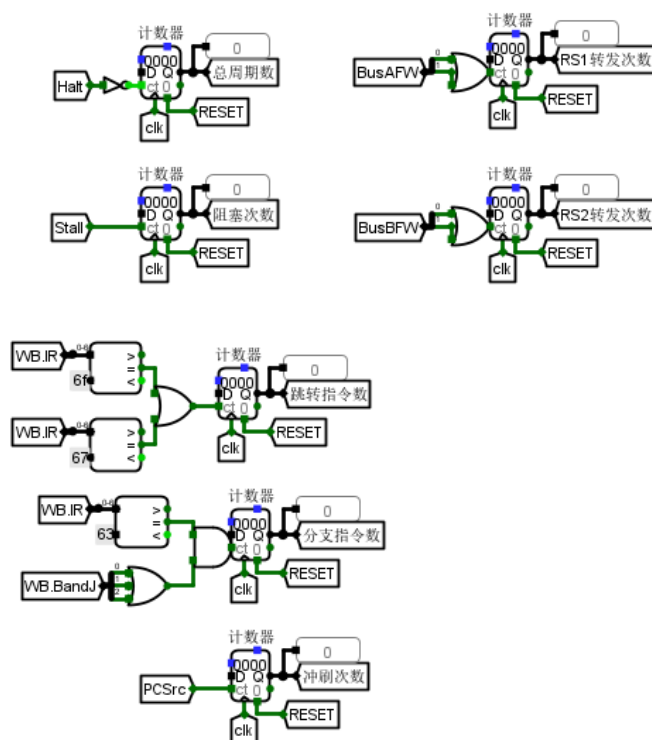


Figure 5: 计数器电路图

## 1.2.5 IFU

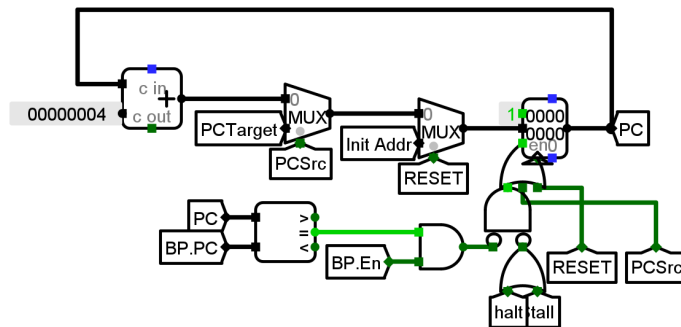
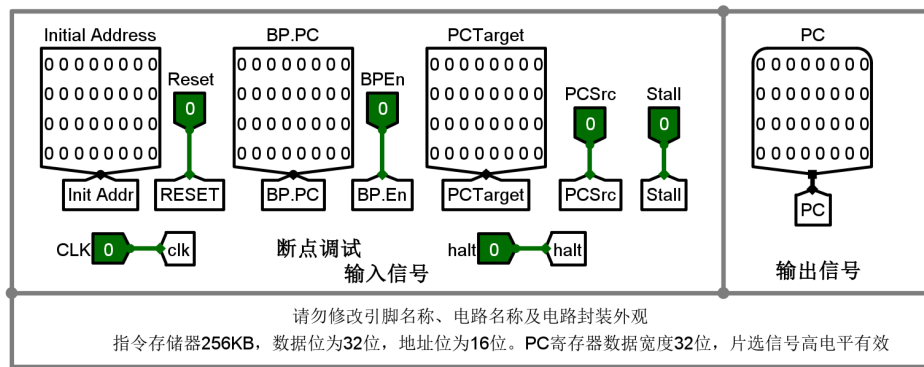


Figure 6: IFU 电路图

## 1.2.6 IDU

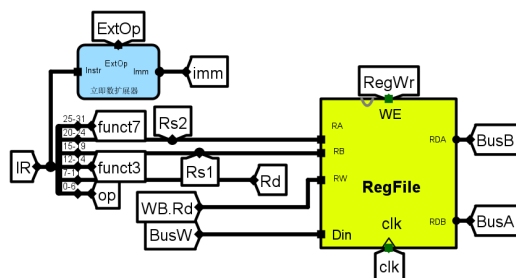
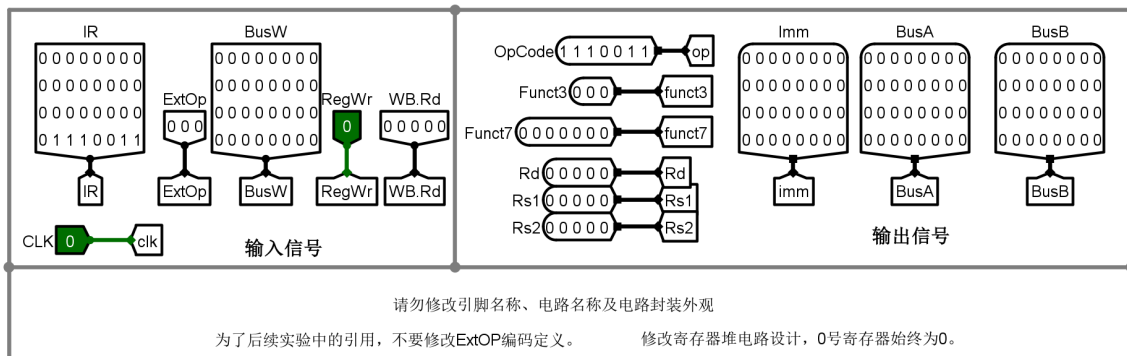


Figure 7: IDU 电路图

## 1.2.7 流水段寄存器

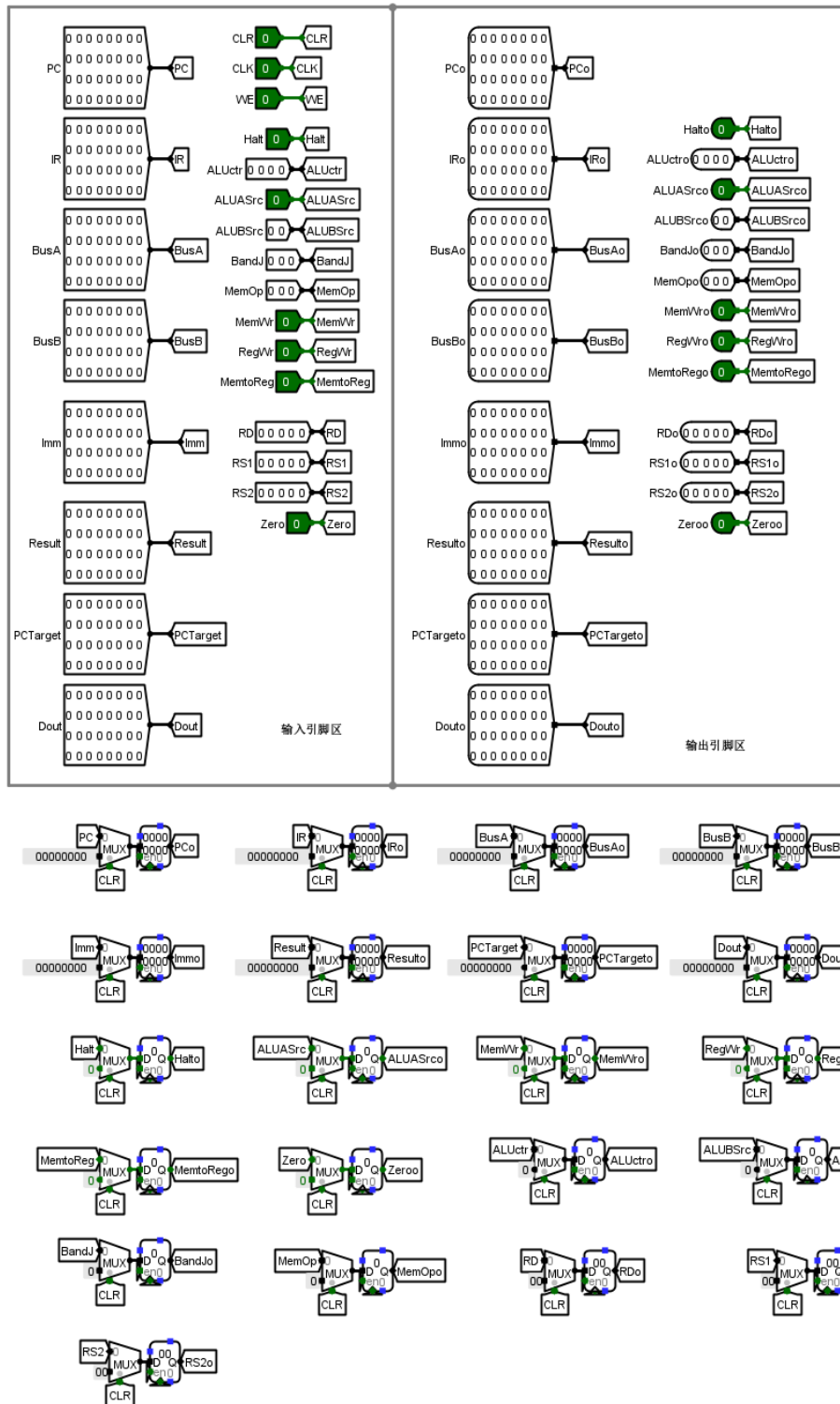


Figure 8: 流水段寄存器电路图

## 1.2.8 EU 模块

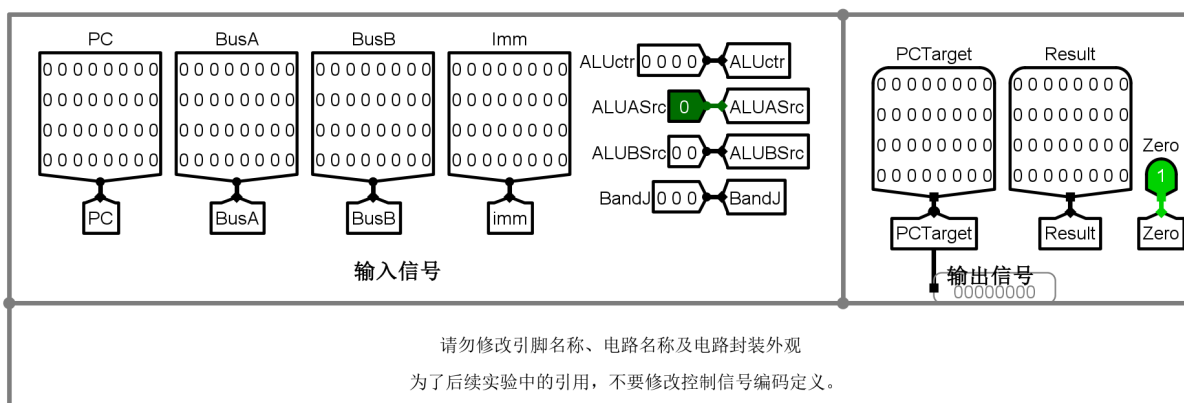


Figure 9: EU 模块电路图

## 1.2.9 CPU 总体设计

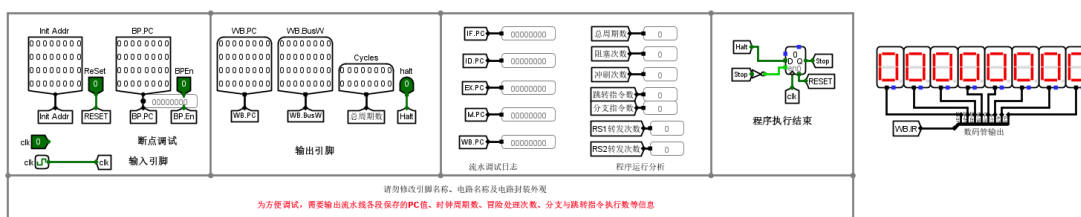


Figure 10: CPU 总体设计电路图

1.3 测试程序验证

给指令存储器加载 lab8.1.hex 文件进行测试，得到如下结果：

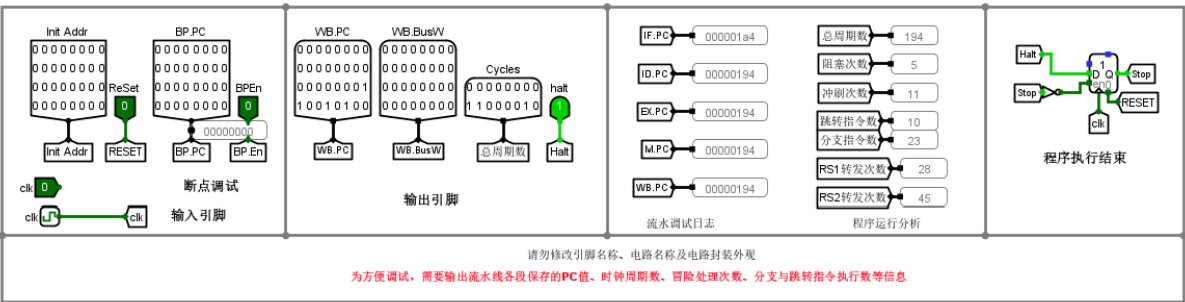


Figure 11: 仿真测试结果

数据如下：

| 计数项目     | 值        |
|----------|----------|
| 3 号寄存器   | 0xe      |
| 10 号寄存器  | 0xc0ffee |
| 12 号寄存器  | 0x59     |
| 总周期数     | 194      |
| 阻塞次数     | 5        |
| 冲刷次数     | 11       |
| 分支指令数    | 10       |
| 跳转指令数    | 23       |
| RS1 转发次数 | 28       |
| RS2 转发次数 | 45       |

Table 1: 测试结果数据表

1.4 遇到的问题

主要问题在于上升沿和下降沿的问题。为了统一，我直接将 main 子电路中 clk 隧道直接连接到时钟上，而不是经过一个反相器。在这个条件下，设置 PC 寄存器、流水段寄存器均为下降沿触发，RegFile 寄存器为上升沿触发。这样可以保证与讲义中描述的时序一致。否则评测时最后一项会少 1。

2 测试程序优化验证

向 CPU 的指令存储器中加载累加程序 lab8.2.asm 如下：

```
.text
main:
    lw a0,0(x0)          # 从数据区地址单元0x0中读取参数n到寄存器a0; n=0,则退出
    beq a0,x0,fail
    ori a2,x0,1           # 循环变量i, 存放在a2, 初值为1
    xor a3,a3,a3          # 累计和存放在a3, 初值为s=0
loop:
    add a3, a3, a2        # 将a3=a3+i
    bgeu a2, a0, finish  # 若i>=n, 则跳出循环
    addi a2, a2, 1        # i++
    jal x0, loop          # 无条件跳转到loop执行
finish:
    sw a3, 4(x0)          # 将累加结果保存到数据存储器地址0x2004单元
pass:
    lui a0,0xc10
    addi a0,a0,-18        # a0=00c0ffee
    ecall                # opcode=0x73, 程序中执行
fail:
    lui a0,0xdeade
    addi a0,a0,-339       # a0=deaddead
    ecall                # opcode=0x73, 程序中执行
```

在数据存储器 DataRAM 地址单元 0x0 中设置初始参数 0x64，测试结果如下：

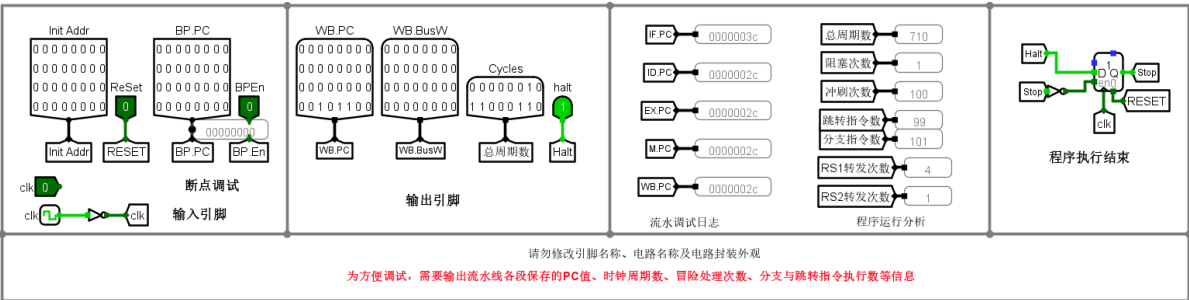


Figure 12: 仿真测试结果

| 计数项目     | 值   |
|----------|-----|
| 总周期数     | 710 |
| 阻塞次数     | 1   |
| 冲刷次数     | 100 |
| 分支指令数    | 99  |
| 跳转指令数    | 101 |
| RS1 转发次数 | 4   |
| RS2 转发次数 | 1   |

Table 2: 测试结果数据表

对上述代码进行优化，修改分支语句判断条件，并删除无条件跳转语句，优化后的代码如下：



```
#计算累加和程序lab8.2-1.asm
.text
main:
    lw a0,0(x0)          # 从数据区地址单元0x0中读取参数n到寄存器a0; n=0,则退出
    beq a0,x0,fail
    ori a2,x0,1           # 循环变量i, 存放在a2, 初值为1
    xor a3,a3,a3          # 累计和存放在a3, 初值为s=0
loop:
    add a3, a3, a2        # 将a3=a3+i
    addi a2, a2, 1        # i++
    bgeu a0,a2,loop       # 若n>=i, 则循环
    sw a3, 4(x0)          # 将累加结果保存到数据存储器地址0x2004单元

pass:
    lui a0,0xc10
    addi a0,a0,-18        # a0=00c0ffee
    ecall                 # opcode=0x73, 程序中中止执行

fail:
    lui a0,0xdeade
    addi a0,a0,-339       # a0=deaddead
    ecall                 # opcode=0x73, 程序中中止执行
```

测试结果如下:

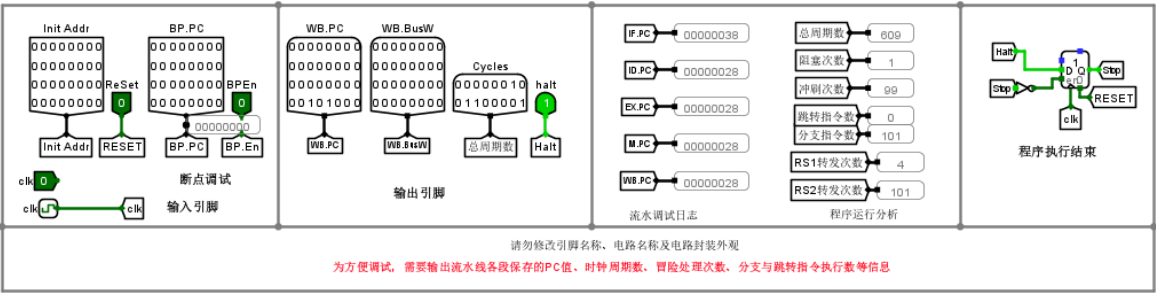


Figure 13: 仿真测试结果

| 计数项目     | 值   |
|----------|-----|
| 总周期数     | 609 |
| 阻塞次数     | 1   |
| 冲刷次数     | 99  |
| 分支指令数    | 101 |
| 跳转指令数    | 0   |
| RS1 转发次数 | 4   |
| RS2 转发次数 | 101 |

Table 3: 测试结果数据表

3 结构优化设计

3.1 优化思路和电路实现

在 M 阶段执行跳转检测，需要冲刷 IF、ID、EX 三个流水段寄存器，IF 阶段需要将 PC 设置为跳转地址。为了减少冲刷带来的时间片损失，在 EX 段执行跳转检测。优化电路图如下：

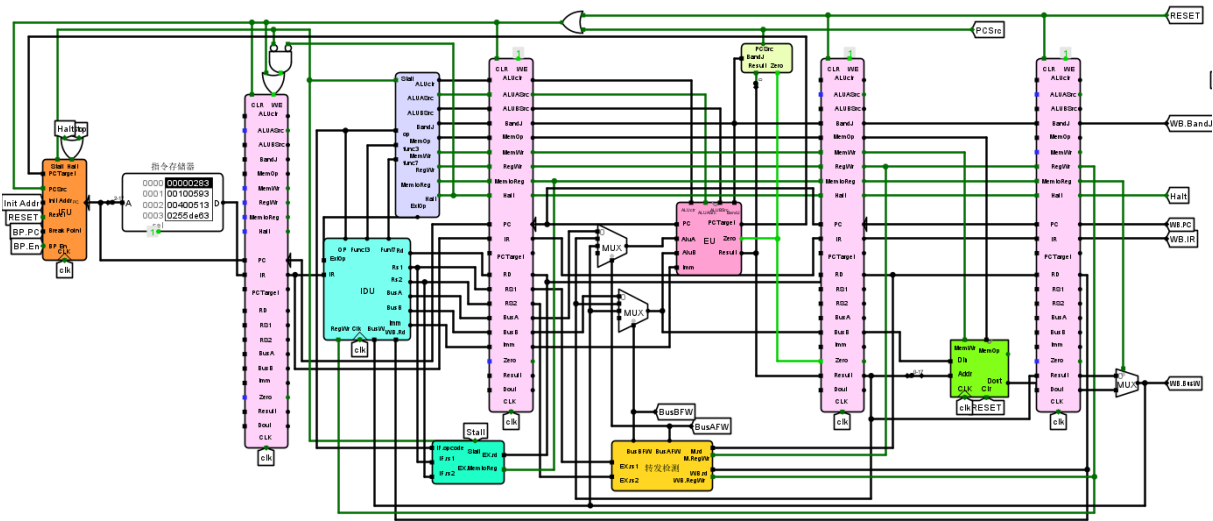


Figure 14: 优化后的 CPU 总体设计电路图

3.2 程序测试

重新加载 lab8.2-1.hex 文件进行测试，得到如下结果：

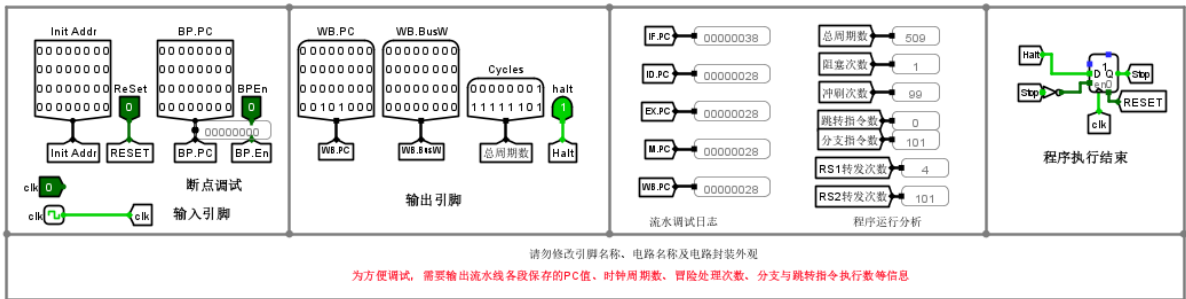


Figure 15: 仿真测试结果

| 计数项目     | 值   |
|----------|-----|
| 总周期数     | 509 |
| 阻塞次数     | 1   |
| 冲刷次数     | 99  |
| 分支指令数    | 101 |
| 跳转指令数    | 0   |
| RS1 转发次数 | 4   |
| RS2 转发次数 | 101 |

Table 4: 测试结果数据表

采用冒泡排序程序进行验证。加载 lab8.3.hex，并在 RAM0 RAM3 中分别加载数据镜像文件 lab8.3.ram0.hex lab8.3.ram3.hex，测试结果如下：

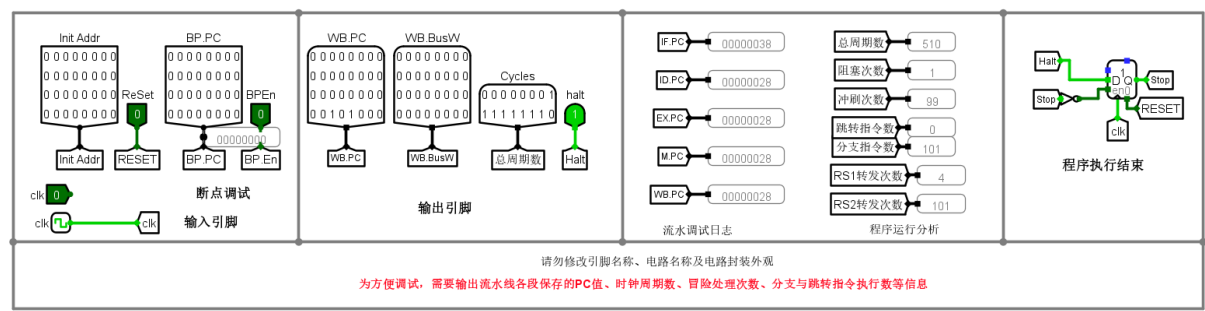


Figure 16: 仿真测试结果

| 计数项目     | 值   |
|----------|-----|
| 总周期数     | 510 |
| 阻塞次数     | 1   |
| 冲刷次数     | 99  |
| 分支指令数    | 101 |
| 跳转指令数    | 0   |
| RS1 转发次数 | 4   |
| RS2 转发次数 | 101 |

Table 5: 测试结果数据表

### 3.3 优化分析

针对前后两次 lab8.2-1.hex 的测试结果进行分析，优化前总周期数为 609，优化后总周期数为 509，减少了 100 个周期。从代码上看，该程序执行了 100 次分支预测失败，每次分支预测失败需要冲刷 3 个流水段寄存器，浪费了 3 个周期。优化后总周期数减少了 100 个周期，说明优化后的 CPU 设计是正确的。

## 4 官方测试集测试

将 CPU 增加适量引脚并封装为测试电路，使用自行编写的测试程序进行测试。封装的 test 电路如下：

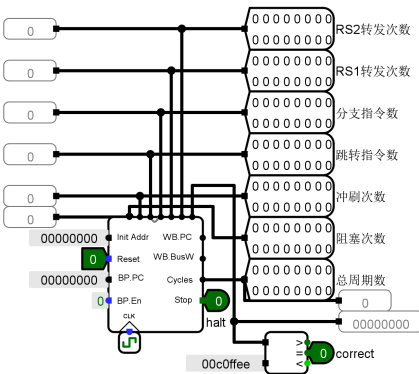


Figure 17: 测试电路

| 测试指令  | RS2 转发<br>次数 | RS1 转发<br>次数 | 分支指令<br>数 | 跳转指令<br>数 | 冲刷次数 | 阻塞次数 | 总周期数 |
|-------|--------------|--------------|-----------|-----------|------|------|------|
| add   | 120          | 84           | 68        | 0         | 16   | 0    | 493  |
| addi  | 47           | 53           | 37        | 0         | 7    | 0    | 252  |
| and   | 92           | 158          | 57        | 0         | 16   | 0    | 513  |
| andi  | 31           | 53           | 26        | 0         | 7    | 0    | 208  |
| auipc | 5            | 5            | 3         | 2         | 3    | 0    | 61   |
| beq   | 59           | 37           | 72        | 0         | 27   | 0    | 341  |
| bge   | 63           | 41           | 81        | 0         | 36   | 0    | 377  |
| bgeu  | 61           | 66           | 81        | 0         | 36   | 0    | 402  |
| blt   | 59           | 37           | 72        | 0         | 27   | 0    | 341  |
| bltu  | 58           | 57           | 72        | 0         | 27   | 0    | 366  |
| bne   | 59           | 38           | 72        | 0         | 29   | 0    | 345  |
| jal   | 5            | 4            | 3         | 2         | 3    | 0    | 57   |
| jalr  | 14           | 27           | 10        | 9         | 13   | 0    | 137  |
| lui   | 9            | 4            | 6         | 0         | 1    | 0    | 63   |
| or    | 92           | 162          | 57        | 0         | 16   | 0    | 516  |
| ori   | 31           | 60           | 26        | 0         | 7    | 0    | 215  |
| sll   | 139          | 94           | 72        | 0         | 16   | 0    | 521  |
| slli  | 51           | 53           | 37        | 0         | 7    | 0    | 251  |
| slt   | 120          | 79           | 68        | 0         | 16   | 0    | 487  |
| slti  | 53           | 48           | 37        | 0         | 7    | 0    | 247  |
| sltiu | 54           | 48           | 37        | 0         | 7    | 0    | 247  |
| sltu  | 120          | 79           | 68        | 0         | 16   | 0    | 487  |
| sra   | 141          | 116          | 73        | 0         | 16   | 0    | 540  |
| srai  | 55           | 73           | 37        | 0         | 7    | 0    | 266  |
| srl   | 141          | 102          | 73        | 0         | 16   | 0    | 534  |
| srli  | 57           | 63           | 37        | 0         | 7    | 0    | 260  |
| sub   | 119          | 82           | 67        | 0         | 16   | 0    | 485  |
| xor   | 92           | 161          | 57        | 0         | 16   | 0    | 515  |
| xori  | 31           | 65           | 26        | 0         | 7    | 0    | 217  |
| lb    | 55           | 79           | 37        | 0         | 7    | 2    | 257  |
| lbu   | 55           | 79           | 37        | 0         | 7    | 2    | 257  |
| lh    | 44           | 83           | 37        | 0         | 7    | 2    | 269  |
| lhu   | 46           | 88           | 37        | 0         | 7    | 2    | 276  |
| lw    | 46           | 89           | 37        | 0         | 7    | 2    | 279  |
| sb    | 84           | 118          | 59        | 0         | 13   | 0    | 452  |
| sh    | 88           | 144          | 59        | 0         | 13   | 0    | 505  |
| sw    | 86           | 153          | 59        | 0         | 13   | 0    | 512  |

测试程序为 autotest.py，但是无法测试 Load 和 Store 指令。

## 5 思考题

### 5.1 转发检测模块中，没有根据操作码 OP 来判断当前指令是否操作 rs1 和 rs2 字段，可能对输出产生什么影响

可能会导致转发检测模块在不需要进行转发的指令上进行转发，如 addi 指令中 imm 段和某些指令的 rs2 段有一定重合，当 rs2 段的值与前面指令的 rd 段值相同并且对应 RegWr 为 1 时，确实会发生转发操作。但是对于结果来说没有影响，因为转发的值实际上不会被采用。没有必要解决该问题。对于被阻塞的指令，由于 ID 段指令全部被清零，所以不会发生转发信号。

### 5.2 本实验中将跳转检测放在 EX 段执行，能否继续优化迁移到 ID 段执行

可行。但是需要考虑下面的问题：

1. 需要在 ID 阶段计算 PCTarget，增加了 ID 阶段的逻辑复杂度，可能会增加 ID 阶段的延迟。
2. 数据冒险。当前 ID 段寄存器中可能存有旧值，需要的额外处理数据转发。
3. 条件分支需要在 ID 段计算条件是否满足，会进一步增加 ID 阶段的逻辑复杂度，可能会增加 ID 阶段的延迟。

### 5.3 讨论三次测试的数据准确性，计算程序的平均 CPI，并讨论提升该测试程序执行效率的方法

#### 5.3.1 数据准确性

上面三个程序测试的数据如下：

| 计数项目     | lab8.2 | lab8.2-1 | lab8.3 |
|----------|--------|----------|--------|
| 总周期数     | 710    | 609      | 510    |
| 阻塞次数     | 1      | 1        | 1      |
| 冲刷次数     | 100    | 99       | 99     |
| 分支指令数    | 99     | 101      | 101    |
| 跳转指令数    | 101    | 0        | 0      |
| RS1 转发次数 | 4      | 4        | 4      |
| RS2 转发次数 | 1      | 101      | 101    |

Table 7: 三种累加和程序测试数据表

上面三个程序均以 100 为参数。

**总周期数**由计数器计数，必然准确。

**阻塞次数**为 1，程序开头的 lw 指令需要等待数据存储器返回数据，阻塞 1 个周期。

**冲刷次数**，lab8.2 中每轮循环的 jal 和 bgeu 指令各一次，共 99+1=100 次冲刷；后续删去 jal 指令后 99 次 bgeu 跳转冲刷。

**RS2 转发次数**，lab8.2 中 bgeu a2,a0,finish 的 rs2 为 a0，首次 lw 写入，后续无需转发，lab8.2-1 中 bgeu a0,a2,loop 的 rs2 为 a2，由上一条 addi 指令更新，每次循环都需要转发，故 101 次转发。

**RS1 转发次数**，先是 lw 和 beq 的数据冒险进行 1 次转发，然后是 xor 指令与 loop 下的第一个 add 指令 1 次转发，pass 下 lui 和 addi 指令进行 1 次转发。多出来的一次是因为阻塞后，ID 段 RS1 值未被清零，导致下一条指令的 RS1 实际上被转发了 2 次。

**分支指令数**，lab8.2 中每轮循环的 bgeu 和 jal 各一次，共 2 次分支；lab8.2-1 中 beq 指令 1 次，每轮循环的 bgeu 一次，共 101 次分支。

**跳转指令数**，lab8.2 中每轮循环的 jal 指令 1 次，共 101 次跳转；lab8.2-1 中删除了 jal 指令，故为 0。

### 5.3.2 计算平均 CPI

$$\begin{aligned}\text{lab8.2: CPI} &= \frac{710}{4 + 400 + 4} = 1.74 \\ \text{lab8.2-1: CPI} &= \frac{609}{4 + 300 + 4} = 1.98 \\ \text{lab8.3: CPI} &= \frac{510}{4 + 400 + 4} = 1.66\end{aligned}$$

### 5.3.3 提升执行效率的方法

1. 减少分支/跳转指令的数量，减少冲刷次数。
2. 提前进行分支/跳转预测。
3. 采用动态分支预测技术。

## 5.4 如何修改流水线 CPU 设计，实现非法指令和结果溢出两种异常处理，标注异常类型，跳转异常处理程序

**非法指令** 检测 opcode、funct3、funct7 等字段，若不匹配任何指令，则标记异常类型为非法指令，跳转到异常处理程序。

**结果溢出** 在 ALU 中增加溢出标志位，检测溢出标志位，若发生溢出，则标记异常类型为结果溢出，跳转到异常处理程序。

在 M 段增加一个异常检测模块，检测到异常时，设置异常标志位，并将异常类型传递给异常处理程序。

当检测到异常时：

1. 清空当前指令和后继指令的流水段寄存器，防止异常指令继续执行。
2. 将该异常指令的 PC 值、异常类型保存并后传到异常检测模块。
3. 异常检测模块将异常类型传递给异常处理程序，并跳转到异常处理程序的入口地址。

## 5.5 实现 1 位动态预测的方法

维护一个 1 位的分支预测器，记录每个分支指令的上一次执行结果（taken 或 not taken）。在 IF 阶段，根据预测结果决定是否取分支。若预测错误，则在 EX 阶段检测到错误后，清空 IF、ID、EX 流水段寄存器，并将 PC 设置为正确的分支目标地址。

## 5.6 高级流水线方法

1. 超流水线：增加流水线级数，使得更多命令可以同时在不同的流水段中执行，提高指令吞吐量。
2. 多发射流水线：在每个时钟周期内发射多条指令，但不一定同时执行，提高指令吞吐量。